

Optimizers, Training Dynamics, Regularization and Generalization

Neural Network Training: The Big Picture

- Training a neural network involves repeatedly performing:
Input Data → Forward Pass → Prediction → Calculate Loss → Backpropagation → Calculate Gradients → Optimizer → Update Weights → Repeat
- Optimization asks: How should the weights be updated? The answer is provided by optimizers.
- Generalization asks: How can the model perform well on unseen data rather than simply memorizing training data? Regularization and appropriate training strategies help address this.

Optimizers

- An optimizer is an algorithm that updates model parameters using gradients calculated during backpropagation.
- The basic update is:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta(\partial L / \partial \theta)$$

where θ is a model parameter,
 L is the loss,
 $\partial L / \partial \theta$ is the gradient, and
 η is the learning rate

- Gradient tells the model which direction to move; learning rate tells the model how large a step to take.

Stochastic Gradient Descent (SGD)

- SGD normally calculates gradients using a mini-batch rather than the entire training dataset.
- Advantages: efficient for large datasets, lower memory requirements than full-batch gradient descent, and frequent parameter updates.
- Limitation: different mini-batches produce slightly different gradients, so the optimization path contains some noise.

Momentum

- Momentum extends SGD by considering previous updates in addition to the current gradient.
- It can reduce oscillation, accelerate convergence, and help the optimizer move consistently toward useful directions.
- A common momentum value is 0.9.
- PyTorch example: `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)`

Adam

- Adam (Adaptive Moment Estimation) combines momentum-like gradient tracking with adaptive scaling of parameter updates.
- Adam can effectively use different step sizes for different parameters.
- A common starting learning rate is 0.001.
- PyTorch example: `optimizer = torch.optim.Adam(model.parameters(), lr=0.001)`
- Adam often converges quickly and is a strong baseline, but SGD with Momentum can sometimes provide better generalization after suitable tuning.

Optimizer Comparison

- SGD: uses mini-batch gradients; common starting LR 0.01–0.1.
- SGD + Momentum: uses current and previous gradients; common starting LR 0.01–0.1.
- Adam: uses momentum-like behavior and adaptive updates; common starting LR 0.001.
- AdamW: Adam with decoupled weight decay; common starting LR 0.001.

These are only starting points, not universal rules.

Learning Rate

- The learning rate controls how much model parameters change during each update.
- If it is too small, training becomes slow and may take a very long time to converge.
- If it is too large, updates may overshoot, causing oscillation or unstable training.
- Choosing an appropriate learning rate is one of the most important parts of neural-network training.

Learning Rate Scheduling

- A learning-rate scheduler changes the learning rate during training. Usually, a relatively larger learning rate is useful early in training, while a smaller rate is useful later for fine adjustment.
- Step Decay reduces the learning rate at fixed intervals.
- Cosine Annealing gradually decreases the learning rate according to a cosine-shaped schedule.
- ReduceLROnPlateau reduces the learning rate when a monitored metric, such as validation loss, stops improving.

Training Dynamics

- Training dynamics describe how the model behaves during training.
- Important quantities include training loss, validation loss, training accuracy, validation accuracy, and learning rate.
- A healthy training process generally shows decreasing loss and improving accuracy, followed by slower progress, learning-rate reduction, and eventual convergence.

Generalization

- Generalization is the ability of a trained model to perform well on unseen data.
- A good model learns general patterns rather than memorizing individual training examples.

Overfitting and Underfitting

- Overfitting occurs when a model performs very well on training data but poorly on unseen data.
Example: training accuracy 99%, validation accuracy 75%.
- Underfitting occurs when the model performs poorly on both training and validation data.
Example: training accuracy 70%, validation accuracy 68%.
- Good generalization is indicated when training and validation performance are both good and reasonably close.

Regularization

- Regularization refers to techniques used to reduce overfitting and improve generalization.
- Important techniques include Dropout, Weight Decay, Data Augmentation, and Batch Normalization, which also has a regularizing effect although its primary purpose is normalization and training stabilization.

Dropout

- Dropout randomly deactivates some neurons during training.
- It prevents the model from becoming too dependent on particular neurons and encourages more robust representations.
- Example: `nn.Dropout(0.5)`, where 0.5 is the dropout probability during training.
- Dropout is active during training and disabled during inference/testing.

Batch Normalization

- Batch Normalization (BatchNorm) normalizes intermediate activations using mini-batch statistics during training.
- Benefits include more stable training, often faster convergence, and some regularization.
- Example: `nn.Conv2d(32, 64, 3)`, `nn.BatchNorm2d(64)`, `nn.ReLU()`.
- Important distinction: Dropout is primarily a regularization technique, while BatchNorm is primarily a normalization and optimization technique.

Weight Decay

- Weight decay discourages the model from developing excessively large weights.
- A common L2-style formulation is: $L_{\text{total}} = L_{\text{data}} + \lambda \sum w_i^2$, where λ controls regularization strength.
- Example: `optimizer = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)`.
- AdamW applies weight decay in a way that is decoupled from Adam's adaptive gradient update.

Data Augmentation

- Data augmentation creates varied versions of existing training samples.
- Common image transformations include horizontal flipping, small rotations, cropping, scaling, translation, and brightness/contrast changes.
- It improves robustness by preventing the model from relying too heavily on specific appearances in the training dataset.

Important Rule for Data Augmentation

- An augmentation should not change the correct class label.
- Only use transformations that are realistic for the particular problem. This is especially important in specialized domains such as medical image classification.

Training vs Validation Data

- Training data can use random augmentation.
- Validation and test data should generally use only consistent preprocessing such as resizing and normalization, not random augmentation.
- This ensures that evaluation represents consistent unseen data.

Combining Optimization and Regularization

- A practical deep-learning model may use several techniques together.
- Example: Data Augmentation → Neural Network with BatchNorm/Dropout → Loss → Backpropagation → AdamW or SGD → Weight Decay → Learning-Rate Scheduler → Updated Parameters.
- These techniques solve different problems: optimizers update parameters; learning rate controls update size; schedulers change the learning rate; Dropout reduces neuron co-adaptation; BatchNorm stabilizes activations; Weight Decay controls large weights; Data Augmentation increases training-data diversity.

Practical Training Example

- If training accuracy is 98% and validation accuracy is 80%, the large gap suggests overfitting.
- Possible actions include realistic data augmentation, appropriate weight decay, dropout where appropriate, monitoring validation performance, and using a learning-rate schedule.
- The objective is to improve validation/generalization performance, not simply increase training accuracy.

Common Problems and Solutions

- Loss decreases very slowly → learning rate may be too small → consider increasing it.
- Loss oscillates → learning rate may be too large → reduce learning rate or use a suitable scheduler.
- Training accuracy is low → possible underfitting → increase model capacity or reduce excessive regularization.
- Training accuracy is very high but validation accuracy is low → possible overfitting → use augmentation, dropout, or weight decay.
- Training is unstable → review learning rate, optimizer, preprocessing, and BatchNorm configuration.
- Validation performance stops improving → consider learning-rate scheduling, regularization, or early stopping.

Key Takeaways

- SGD updates parameters using mini-batch gradients.
- Momentum uses previous update information to make SGD smoother and faster.
- Adam combines momentum-like behavior with adaptive parameter updates.
- Learning rate controls the size of parameter updates.
- Learning-rate schedules adjust the learning rate during training.
- Generalization means performing well on unseen data.
- Overfitting means excellent training performance but poor unseen-data performance.
- Dropout randomly disables neurons during training.
- BatchNorm normalizes activations and stabilizes training.
- Weight decay discourages excessively large weights.
- Data augmentation increases the diversity of training examples.
- Too much regularization can cause underfitting.